

AI Agent for Enzyme and Cell Factory Design

Shanghai Jiao Tong University
Summer Research Programme, Project 127

Hasmik Chilingaryan, University of Birmingham
Chloe Zheng, University of California, San Diego
Anastasiia Savina, Lomonosov Moscow State University

Supervisors:
Prof. Yongyong Shi
Dr. Zhuo Wang, Dr. Kai Yi

Contents

01

BACKGROUND

02

ENZYME KINETICS

03

METABOLIC NETWORK
DESIGN

04

KIMI THE AI AGENT

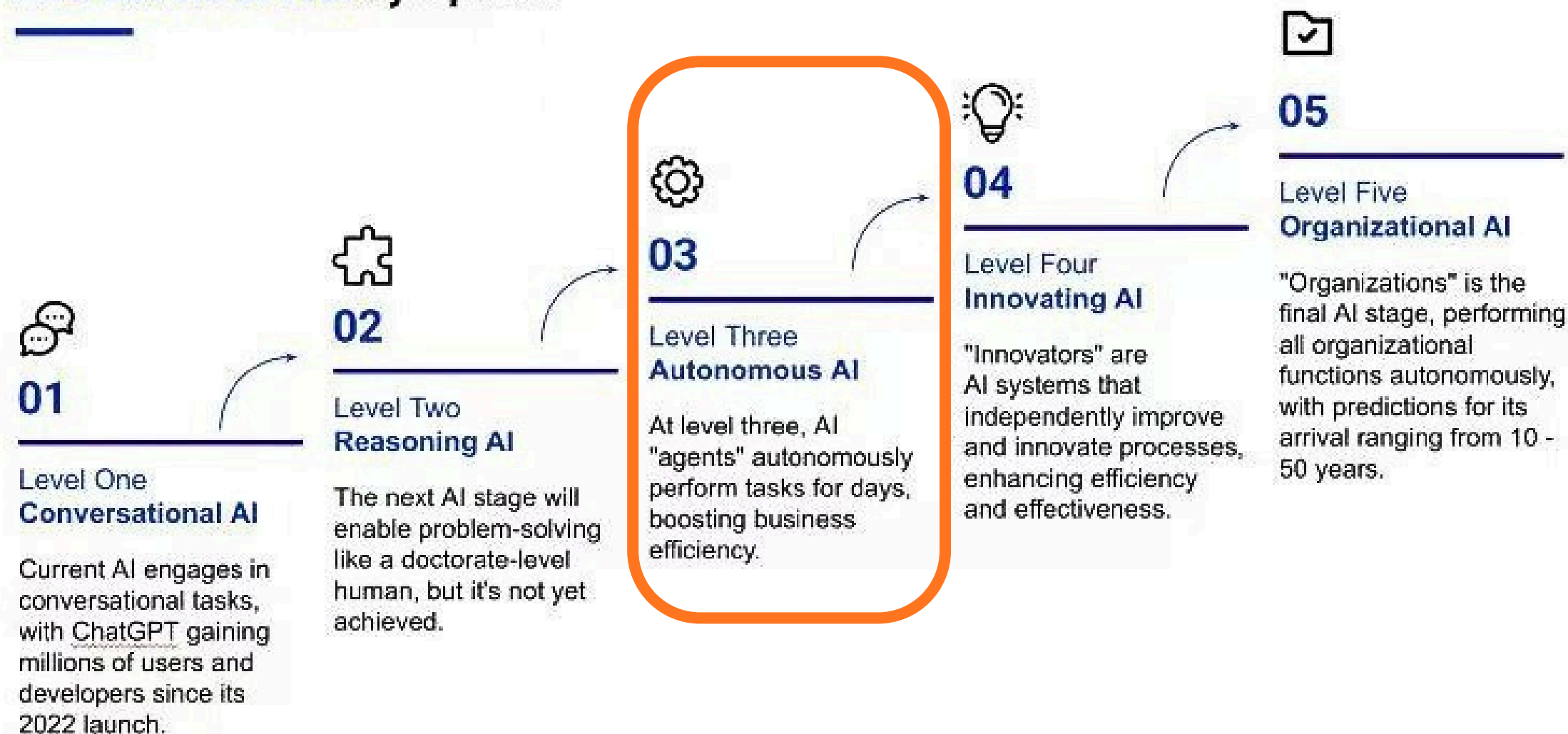


BACKGROUND

SYNTHETIC ENZYME BIOLOGY

The Evolution of AI & Autonomous Agents

Five Levels of AGI by OpenAI

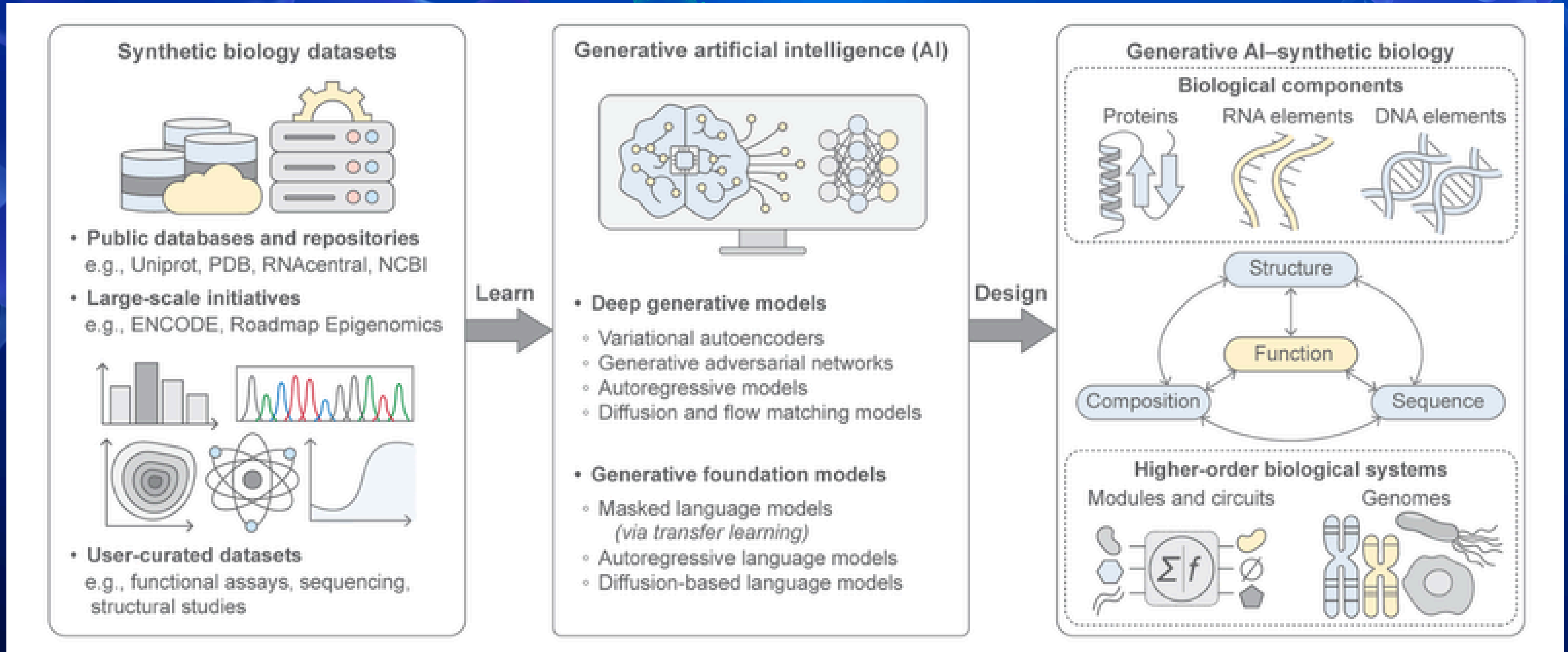


Source: [Forbes Article](#)

Evolving from Level 1 conversational text and level 2 reasoning to level 3 Autonomous Agents capable of tool-calling and executing multi-step scientific workflows independently.

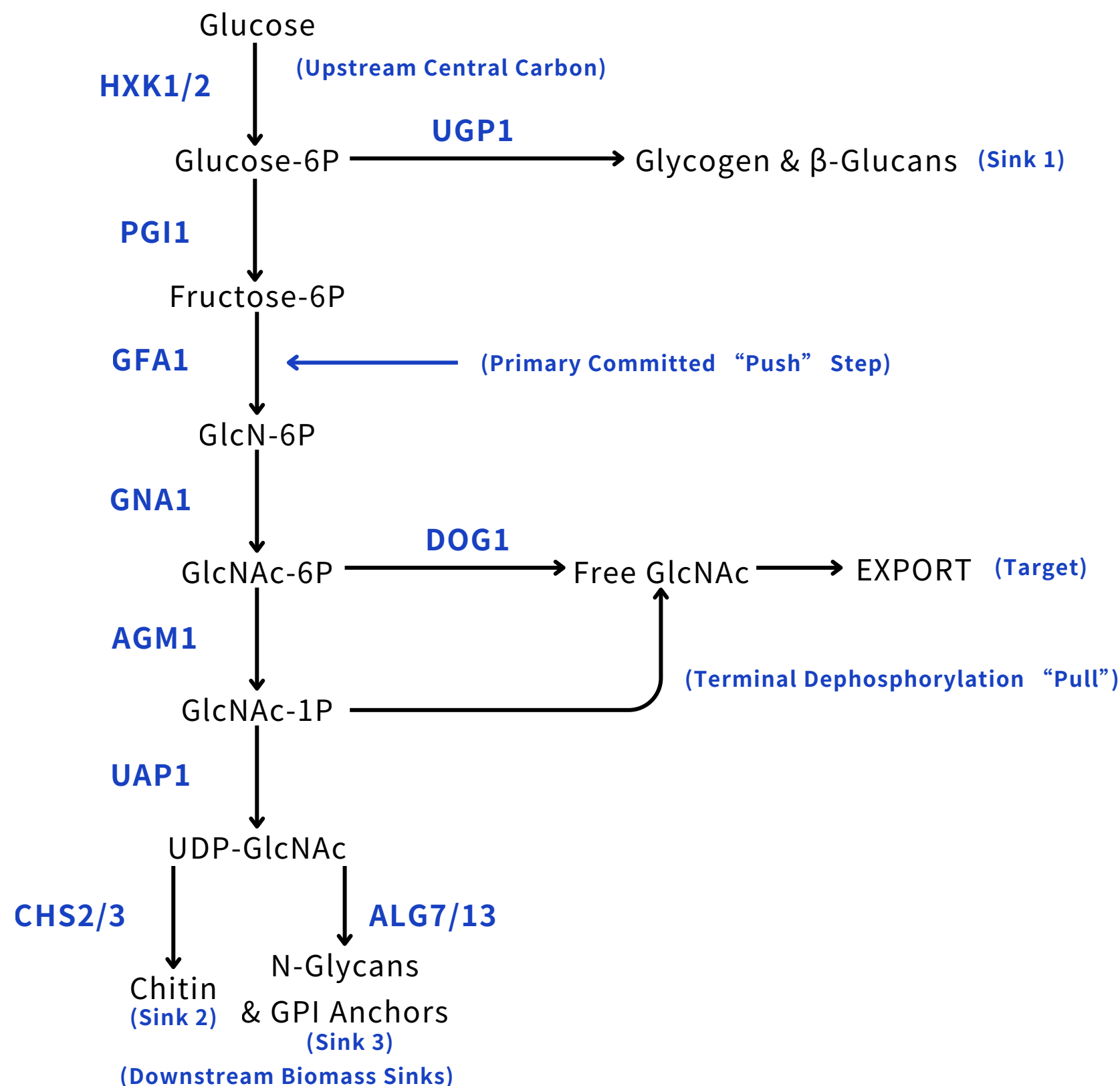
Autonomous agents deliver high impact data-dense biology, navigating vast combinatorial design spaces beyond human trial-and-error.

Generative AI in Synthetic Biology



Benchmark Target: N-Acetylglucosamine (GlcNAc) in Yeast

Glycolysis - GlcNAc Pathway



Target Small Molecule:

N-Acetylglucosamine (GlcNAc), a high-value pharmaceutical and nutraceutical precursor.

Why *S. cerevisiae*?

Industrial low-pH tolerance, robust fermentation characteristics, and high eukaryotic protein expression capacity.

The Metabolic Bottleneck

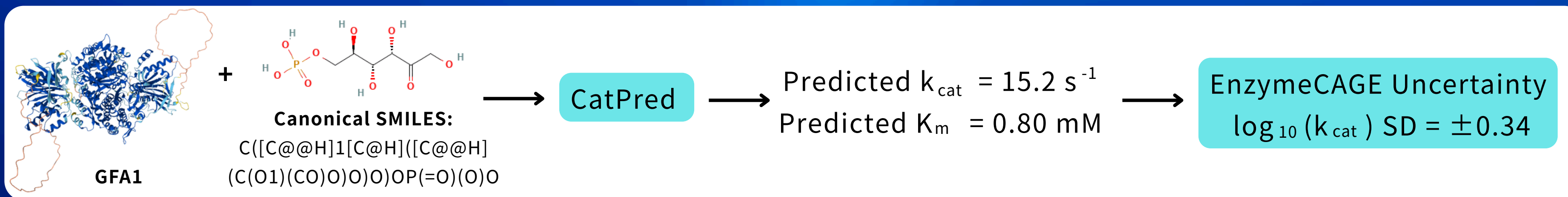
In wild-type yeast, aminosugars are strictly allocated toward structural cell wall biosynthesis rather than excreted as free monomeric GlcNAc.



ENZYME KINETIC PIPELINE

DEEP LEARNING & STRUCTURAL MODELS

Deep Learning-Driven Kinetic Parameterisation



Automated Data Harvesting

UniProt ID Extraction, FASTA sequence acquisition, and SMILES canonical sanitisation

Sequence & Structural Embeddings

ESM-2 protein language embeddings combined with RDKit molecular Morgan fingerprints

Kinetic Predictions

Utilising CatPred to infer turnover numbers (k_{cat}) and Michaelis-Menten constants (K_m)

Uncertainty Quantification

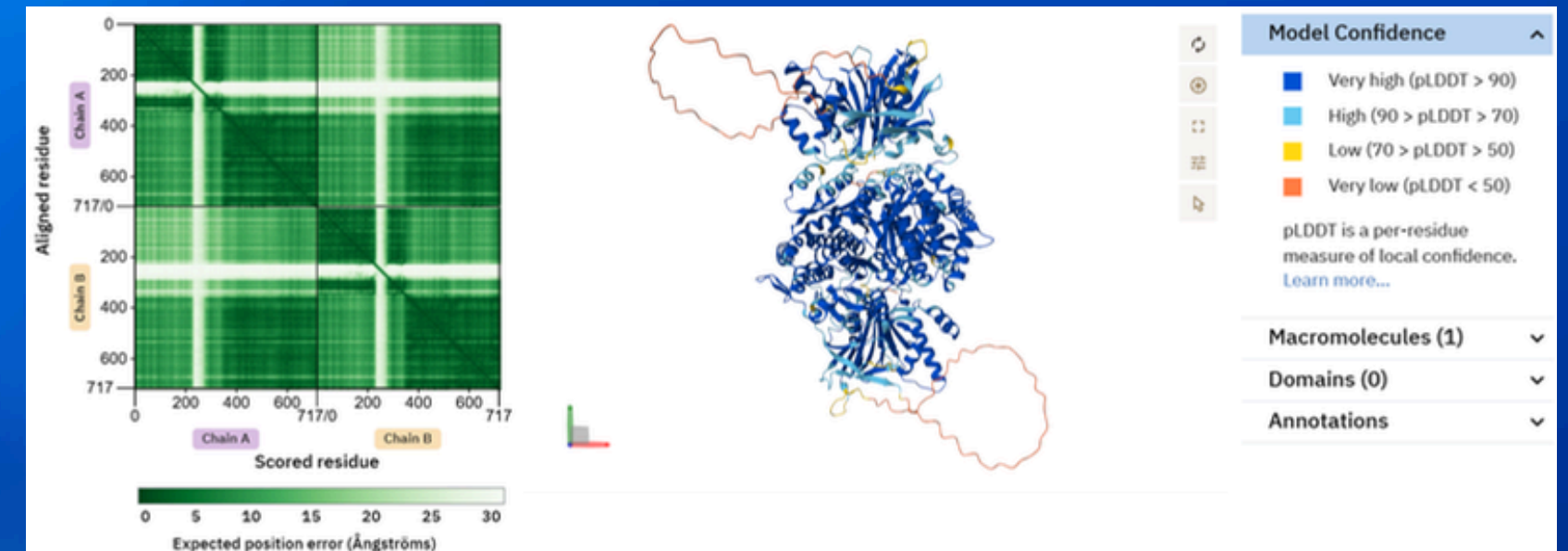
EnzymeCAGE generates predictive standard deviations (SD) to quantify parameter confidence before feeding into metabolic simulations

```
Row 1 (HXK2): kcat=98.02 s^-1 | Km=1.02 mM | pLDDT=96.04 (PASS (High Confidence))
Row 2 (HXK1): kcat=132.32 s^-1 | Km=0.756 mM | pLDDT=95.81 (PASS (High Confidence))
Row 3 (PGI1): kcat=120.3 s^-1 | Km=0.831 mM | pLDDT=97.04 (PASS (High Confidence))
Row 4 (UGP1): kcat=3332.74 s^-1 | Km=0.03 mM | pLDDT=90.89 (PASS (High Confidence))
Row 5 (GFA1): kcat=43.73 s^-1 | Km=2.287 mM | pLDDT=87.87 (PASS (High Confidence))
Row 6 (GNA1): kcat=676.79 s^-1 | Km=0.148 mM | pLDDT=97.65 (PASS (High Confidence))
Row 7 (AGM1): kcat=2081.46 s^-1 | Km=0.048 mM | pLDDT=95.49 (PASS (High Confidence))
Row 8 (UAP1): kcat=231.42 s^-1 | Km=0.432 mM | pLDDT=94.97 (PASS (High Confidence))
Row 9 (CHS3): kcat=1312.18 s^-1 | Km=0.076 mM | pLDDT=74.38 (PASS (High Confidence))
Row 10 (CHS2): kcat=567.19 s^-1 | Km=0.176 mM | pLDDT=72.79 (PASS (High Confidence))
Row 11 (GPI3): kcat=237.91 s^-1 | Km=0.42 mM | pLDDT=89.66 (PASS (High Confidence))
Row 12 (GPI12): kcat=1622.91 s^-1 | Km=0.062 mM | pLDDT=89.84 (PASS (High Confidence))
Row 13 (ALG7): kcat=1388.95 s^-1 | Km=0.072 mM | pLDDT=90.27 (PASS (High Confidence))
Row 14 (ALG13): kcat=8297.47 s^-1 | Km=0.012 mM | pLDDT=71.37 (PASS (High Confidence))
Row 15 (DOG1): kcat=849.99 s^-1 | Km=0.118 mM | pLDDT=95.45 (PASS (High Confidence))
```

AlphaFold pLDDT Structural Confidence Gate

```
def calculate_mean_plddt(pdb_path):  
    """Extracts average pLDDT score from AlphaFold PDB B-factor column."""  
    if not pdb_path or not os.path.exists(pdb_path):  
        return None  
    plddt_scores = []  
    with open(pdb_path, "r") as f:  
        for line in f:  
            if line.startswith("ATOM") or line.startswith("HETATM"):  
                try:  
                    b_factor = float(line[60:66].strip())  
                    plddt_scores.append(b_factor)  
                except ValueError:  
                    continue  
    return round(sum(plddt_scores) / len(plddt_scores), 2) if plddt_scores else None
```

- B-factor column extraction
 - Threshold: pLDDT ≥ 70.0
 - 100% Pathway Enzymes
- Pass rate (15/15)

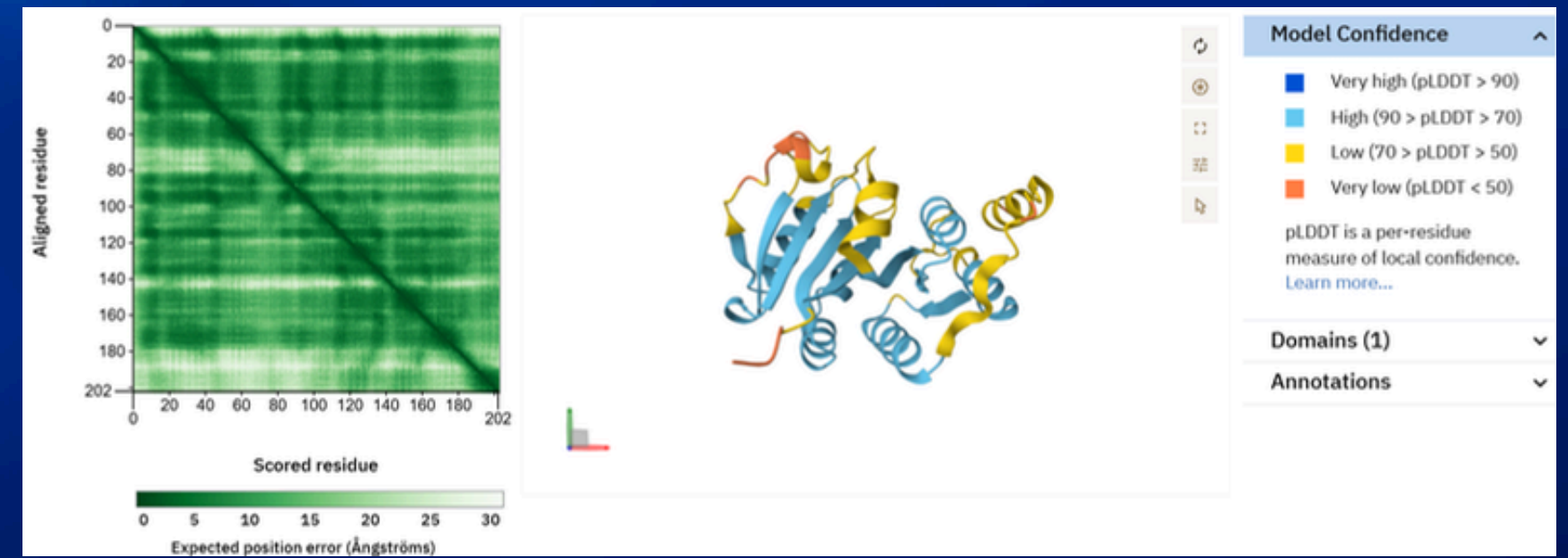


GFA1 (UniProt: P14742)

Mean pLDDT: 86.4

Status: PASS (High Confidence)

- Well-folded catalytic core
- Feeds into ecModel



ALG13 (UniProt: P53178)

Mean pLDDT: 71.4

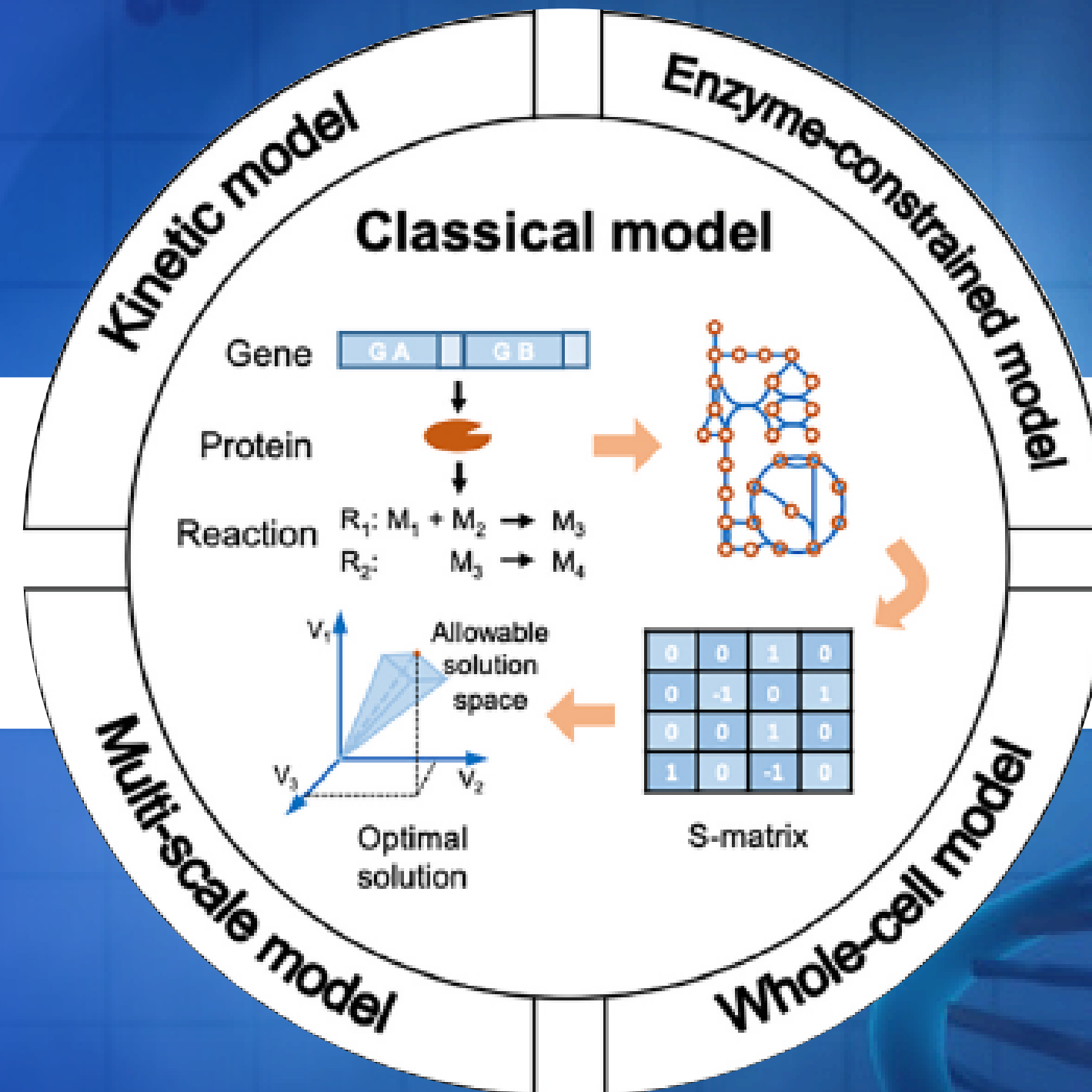
Status: Pass (High Confidence)

- Structural disorder present

Bridge Molecular Kinetics to ecModels

The Limitation of Classical FBA

- Stoichiometry Only
- Unrealistic Phenotypes



The ecGEM Solution

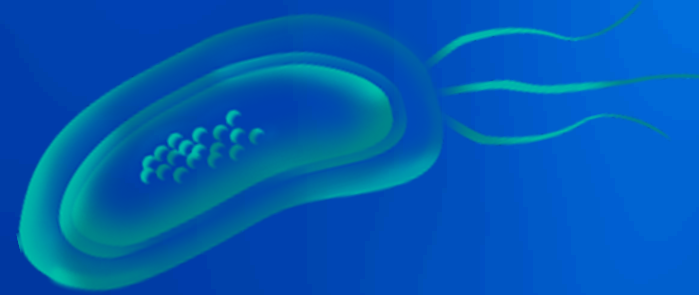
- Real Catalytic Constraints
- Total Protein Pool
- Direct Integration



NETWORK

From GECKO MATLAB to a Python based approach

Introduction to the method: GECKO



What it is used for:

- Builds enzyme-constrained metabolic models (ecModels)
 - Adds real k_{cat} (enzyme speed) and protein cost to every reaction
 - Lets us predict which enzyme is limiting production, not just what's chemically possible

**GECKO = enhances a
Genome-scale
metabolic model with
Enzymatic Constraints,
using Kinetic and
Omics data.**

Why it is relevant:

- Connects Task 1's predicted enzyme kinetics directly to a working cell model
 - Finds real bottlenecks grounded in enzyme speed, not just network topology
- Same method validated in the steroid production paper we're benchmarking against



What the GECKO Pipeline Built:

Enzyme-constrained yeast model

Built via GECKO 3.0 in MATLAB - every reaction limited by real enzyme kcat and protein availability, not just stoichiometry.

Real kinetics applied

Hasmik' s predicted kcat values applied to the GlcNAc pathway, rest of the network covered by BRENDA/standard fallback kcats.

Three callable tools

build_model(), find_bottleneck(), suggest_targets() - the actual interface the AI agent is meant to call.

build_model() – what it does

- Maps a target name (e.g. 'GlcNAc') to its pre-built model files.
- Returns both models plus a metadata bundle (product reaction, max growth rate) so the caller never needs to know internal file names.

```
function [ecModel, ecModelFull, meta] = build_model(targetName, kcatFilePath)
switch targetName
case 'GlcNAc'
    lightFile = 'ecGlcNAc_light_functional.yml';
    fullFile = 'ecGlcNAc_full_functional.yml';
    productRxn = 'r_GLCNAC_EXCHANGE';
otherwise
    error('build_model: unknown targetName "%s". Expected "GlcNAc".', targetName);
end

% --- apply Student A's kcat file ---
params = ModelAdapterManager.getDefault().getParameters();
destPath = fullfile(params.path, 'data', 'customKcats.tsv');
if ~strcmp(kcatFilePath, destPath)
    try
        copyfile(kcatFilePath, destPath);
    catch ME
        if ~contains(ME.message, 'same')
            rethrow(ME);
        end
    end
end

% --- build light model ---
ecModel = loadEcModel(lightFile);
ecModel = applyCustomKcats(ecModel);
ecModel = applyKcatConstraints(ecModel);
ecModel = setProtPoolSize(ecModel);

% --- build full model ---
ecModelFull = loadEcModel(fullFile);
ecModelFull = applyCustomKcats(ecModelFull);
ecModelFull = applyKcatConstraints(ecModelFull);
ecModelFull = setProtPoolSize(ecModelFull);

% --- package everything the agent needs ---
meta.targetName = targetName;
meta.productRxn = productRxn;
meta.maxGrowth = getMaxGrowth(ecModelFull);
meta.lightFile = lightFile;
meta.fullFile = fullFile;

end
```

find_bottleneck() – what it does

```
function usageReport = find_bottleneck(ecModel, growthRate)

params = ModelAdapterManager.getDefault().getParameters();
ecModel = setParam(ecModel, 'lb', params.bioRxn, growthRate);
ecModel = setParam(ecModel, 'obj', params.c_source, 1);
sol = solveLP(ecModel, 1);
if sol.stat ~= 1
    error(['find_bottleneck: LP solve failed (infeasible) at growth rate '
        '%.5f. Try a lower value.'], growthRate);
end
usageData = enzymeUsage(ecModel, sol.x);
usageReport = reportEnzymeUsage(ecModel, usageData);

end
```

- Fixes model at a target growth rate.
- Minimizes parsimonious FBA.
- Computes how much of the protein budget each enzyme consumes.
- Returns enzymes ranked by usage – top row = limiting bottleneck.
- What do these results mean?

protID	geneID	absUsage	percUsage	kcat	source	rxnID	rxnNames
{'P36148'}	{'YKR067W'}	5.2858	4.2287	NaN	{'=== '}	{'=== '}	{'involved in multiple rxns, usage combined, individual rxns below' }
{'P36148'}	{'YKR067W'}	2.3999	1.9199	0.0918	{'brenda' }	{'r_2386_EXP_2'}	{'glycerol-3-phosphate acyltransferase (16:0), ER membrane' }
{'P36148'}	{'YKR067W'}	2.2797	1.8237	0.0918	{'brenda' }	{'r_2389_EXP_2'}	{'glycerol-3-phosphate acyltransferase (16:1), ER membrane' }
{'P36148'}	{'YKR067W'}	0.60631	0.48505	0.0918	{'brenda' }	{'r_2310' }	{'glycerol-3-phosphate acyltransferase (18:0), ER membrane' }
{'P05694'}	{'YER091C'}	4.8163	3.8531	0.33	{'brenda' }	{'r_0727' }	{'methionine synthase' }
{'P07251'}	{'YBL099W'}	4.4687	3.575	20	{'brenda' }	{'r_0226' }	{'ATP synthase' }
{'P00830'}	{'YJR121W'}	4.1779	3.3423	20	{'brenda' }	{'r_0226' }	{'ATP synthase' }
{'P06168'}	{'YLR355C'}	3.5653	2.8523	NaN	{'=== '}	{'=== '}	{'involved in multiple rxns, usage combined, individual rxns below' }
{'P06168'}	{'YLR355C'}	2.5613	2.0491	0.98	{'brenda' }	{'r_0096' }	{'acetoxy acid isomerase' }
{'P06168'}	{'YLR355C'}	1.004	0.80318	0.98	{'brenda' }	{'r_0669' }	{'ketol-acid reductoisomerase (2-aceto-3-hydroxybutanoate)' }
{'Q04728'}	{'YHR062C'}	3.5298	2.8238	0.22	{'brenda' }	{'r_0818' }	{'ornithine transacetylase' }
{'P33312'}	{'YBR153W'}	3.3974	2.7179	0.0006667	{'brenda' }	{'r_0015' }	{'2,5-diamino-6-ribosylamino-4(3H)-pyrimidinone 5'-phosphate reductase (NADPH)' }
{'P00401'}	{'Q0045' }	3.3838	2.643	48	{'brenda' }	{'r_0438_EXP_4'}	{'ferrocyanide-cytochrome c oxidoreductase' }
{'P38877'}	{'YBR039W'}	2.6192	2.0953	20	{'brenda' }	{'r_0226' }	{'ATP synthase' }
{'P38972'}	{'YGR061C'}	2.5474	2.0379	0.5	{'sensitivityTuning'}	{'r_0079' }	{'5'-phosphoribosylformyl glycylamide synthetase' }

suggest_targets() – what it does

- Scans 10 growth rates between 10–90% of the models max.
- At each rate, maximizes flux toward the product exchange reaction.
- Compares flux dist at the two extremes
- Returns the 10 rxns with the largest flux change – candidate overexpression/KO targets.
- What do these results mean?

```
function targets = suggest_targets(ecModel, productRxn)

params = ModelAdapterManager.getDefault().getParameters();
rxnIdx = getIndexes(ecModel, productRxn, 'rxns');
if isempty(rxnIdx)
    error('suggest_targets: reaction "%s" not found in this ecModel.', productRxn);
end

mGrowth = setParam(ecModel, 'obj', params.bioRxn, 1);
solGrowth = solveLP(mGrowth);
maxGrowth = solGrowth.x(getIndexes(mGrowth, params.bioRxn, 'rxns'));

growthRates = linspace(0.1*maxGrowth, 0.9*maxGrowth, 10);
fluxTable = zeros(numel(ecModel.rxns), numel(growthRates));

for i = 1:numel(growthRates)
    m = setParam(ecModel, 'lb', params.bioRxn, growthRates(i));
    m = setParam(m, 'obj', productRxn, 1);
    sol = solveLP(m);
    if sol.stat ~= 1
        error('suggest_targets: infeasible at growth rate %.5f.', growthRates(i));
    end
    fluxTable(:,i) = sol.x;
end

slopes = fluxTable(:,end) - fluxTable(:,1);
[~, idx] = sort(abs(slopes), 'descend');
targets = ecModel.rxns(idx(1:10));

end
```

ans =

10x1 [cell](#) array

```
{'prot_pool_exchange'      }
{'carbon dioxide exchange' }
{'CO2 transport'           }
{'glyceraldehyde-3-phosphate dehydrogenase'}
{'phosphoglycerate kinase'  }
{'ethanol exchange'        }
{'ethanol transport'       }
{'enolase'                 }
{'phosphoglycerate mutase'  }
{'pyruvate decarboxylase'   }
```

Why GECKO did not work...

Matlab enviornment on the shared server

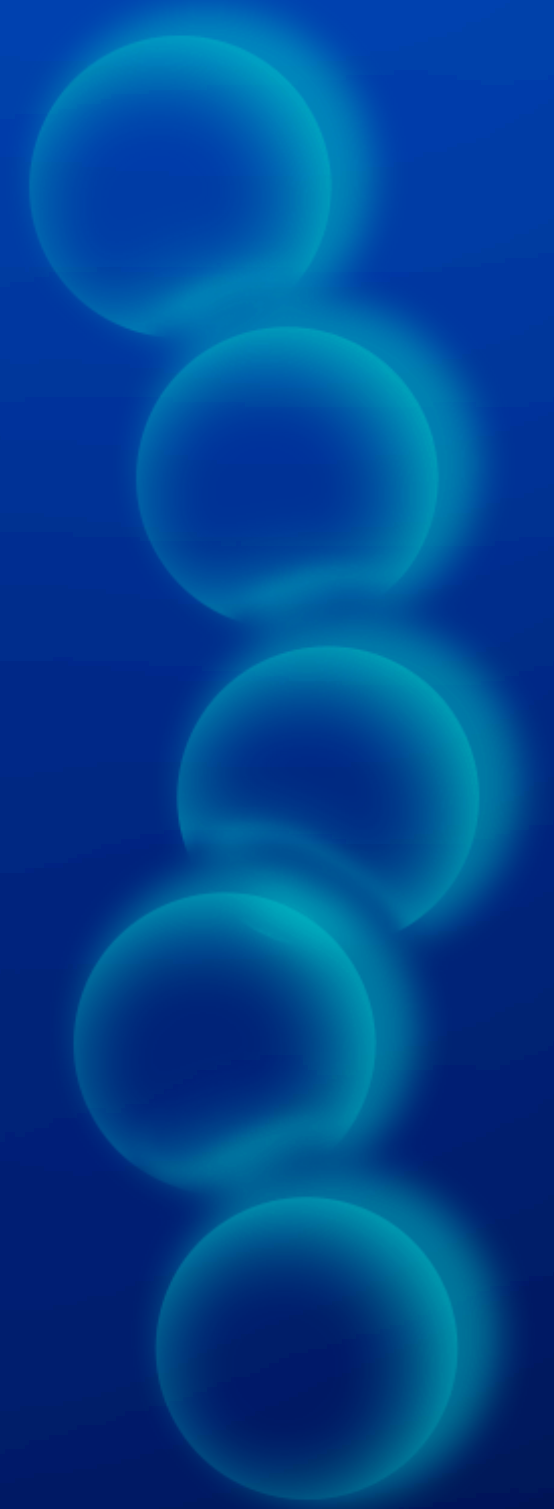
The agent (running on our shared server) could not reliably launch or call into a MATLAB/GECKO session — licensing, environment setup, and reproducibility all became blockers.

Matlab code VS Python

Our server relied on Python code and it was difficult to translate code files from Matlab to Python.

MEWpy!

GECKO/MATLAB was hard to integrate – a Python-native option (MEWpy) could offer the same enzyme-constrained modeling – transition.



New Python Pipeline

The same enzyme-constrained modeling concept, but instead, rebuilt natively in Python.

build_glcnac_model.py – Loads an enzyme-constrained yeast model and applies real kcat data

find_bottleneck.py – Identifies the current rate-limiting enzyme(s)

suggest_targets.py – Enzyme-aware search for engineering targets

build_glcnac_model.py

```
import csv
from mewpy.model.gecko import GeckoModel
from mewpy.simulation import get_simulator

UDPGLCNAC_MET = 's_1544_c'
UDP_MET = 's_1538_c'
GLCNAC_MET = 's_4017_c'
DEPHOSPHO_RXN_ID = 'r_GLCNAC_1'
EXPORT_RXN_ID = 'r_GLCNAC_EXCHANGE'
BIOMASS_RXN = 'r_2111'
KCAT_FILE = 'data/customKcats_GlcNac.tsv'

def load_kcat_data(path=KCAT_FILE):
    rows = []
    with open(path) as f:
        reader = csv.DictReader(f, delimiter='\t')
        for row in reader:
            if row['structure_status'] == 'REJECT':
                print(f" [SKIPPED - low confidence] {row['gene']} ({row['uniprot']}): {row['notes']}")
                continue
            rows.append(row)
    return rows

def build_glcnac_model():
    model = GeckoModel('single-pool')
    sim = get_simulator(model)

    sim.add_metabolite(
        GLCNAC_MET, formula='C8H15NO6',
        name='N-Acetyl-D-glucosamine [cytoplasm]', compartment='c',
    )

    sim.add_reaction(
        rxn_id=DEPHOSPHO_RXN_ID,
        name='UDP-GlcNac dephosphorylation (unconstrained)',
        stoichiometry={UDPGLCNAC_MET: -1, GLCNAC_MET: 1, UDP_MET: 1},
        reversible=False, lb=0, ub=1000, gpr=None,
    )

    sim.add_reaction(
        rxn_id=EXPORT_RXN_ID,
        name='GlcNac exchange',
        stoichiometry={GLCNAC_MET: -1},
        reversible=True, lb=-1000, ub=1000, gpr=None,
    )
```

```
print("Applying real kcat values:")
kcat_rows = load_kcat_data()
applied = []
not_found = []
for row in kcat_rows:
    uniprot = row['uniprot']
    new_kcat = float(row['kcat_s1'])
    try:
        existing = sim.get_Kcats(uniprot)
    except ValueError:
        not_found.append((row['gene'], uniprot))
        continue
    for rxn_id in existing:
        if rxn_id.startswith('draw_prot_'):
            continue
        sim.set_Kcat(uniprot, rxn_id, new_kcat)
        applied.append((row['gene'], uniprot, rxn_id, new_kcat))

for gene, uniprot, rxn_id, kcat in applied:
    print(f" [APPLIED] {gene} ({uniprot}) -> {rxn_id}: kcat = {kcat}")
for gene, uniprot in not_found:
    print(f" [NOT FOUND IN MODEL] {gene} ({uniprot})")

return sim

__name__ == '__main__':
    sim = build_glcnac_model()
    print(f"\n{DEPHOSPHO_RXN_ID} present:", DEPHOSPHO_RXN_ID in sim.reactions)
    print(f"{EXPORT_RXN_ID} present:", EXPORT_RXN_ID in sim.reactions)

    sim.objective = BIOMASS_RXN
    result = sim.simulate()
    print(f"Growth rate: {result.objective_value:.5f} /hour")
```

```
[(venv) ubuntu@VM-0-12-ubuntu:~/AISynbio/task2_python$ python build_glcnac_model.py
Applying real kcat values:
[APPLIED] HXK2 (P04807) -> r_0533No3: kcat = 250.0
[APPLIED] HXK2 (P04807) -> r_0534No4: kcat = 250.0
[APPLIED] HXK2 (P04807) -> r_0535No3: kcat = 250.0
[APPLIED] HXK1 (P04806) -> r_0533No2: kcat = 280.0
[APPLIED] HXK1 (P04806) -> r_0534No3: kcat = 280.0
[APPLIED] HXK1 (P04806) -> r_0535No2: kcat = 280.0
[APPLIED] PGI1 (P12709) -> r_0467No1: kcat = 1200.0
[APPLIED] PGI1 (P12709) -> r_0467_REVNo1: kcat = 1200.0
[APPLIED] UGP1 (P32861) -> r_1084No2: kcat = 310.0
[APPLIED] UGP1 (P32861) -> r_1084_REVNo2: kcat = 310.0
[APPLIED] GFA1 (P14742) -> r_0477No2: kcat = 15.2
[APPLIED] GNA1 (P43577) -> r_0760No1: kcat = 45.0
[APPLIED] GNA1 (P43577) -> r_0760_REVNo1: kcat = 45.0
[APPLIED] AGM1 (P38628) -> r_0882No1: kcat = 88.0
[APPLIED] AGM1 (P38628) -> r_0890No1: kcat = 88.0
[APPLIED] AGM1 (P38628) -> r_0882_REVNo1: kcat = 88.0
[APPLIED] AGM1 (P38628) -> r_0890_REVNo1: kcat = 88.0
[APPLIED] UAP1 (P43123) -> r_1069No1: kcat = 110.0
[APPLIED] UAP1 (P43123) -> r_1069_REVNo1: kcat = 110.0
[APPLIED] CHS3 (P29465) -> r_0272No1: kcat = 3.5
[APPLIED] CHS2 (P14180) -> r_0272No2: kcat = 5.1
[APPLIED] GPI12 (P23797) -> r_0518No1: kcat = 18.0
[APPLIED] GPI12 (P23797) -> r_0518_REVNo1: kcat = 18.0
[NOT FOUND IN MODEL] GPI3 (P32363)
[NOT FOUND IN MODEL] ALG7 (P07286)
[NOT FOUND IN MODEL] ALG13 (P53178)
[NOT FOUND IN MODEL] DOG1 (P38774)

r_GLCNAC_1 present: True
r_GLCNAC_EXCHANGE present: True
Growth rate: 0.26205 /hour
```

Loads an enzyme-constrained yeast model directly in Python.

find_bottleneck.py

```
from build_glcnac_model import build_glcnac_model, BIOMASS_RXN, EXPORT_RXN_ID

# Same 10 real, model-present proteins used in suggest_targets.py
CANDIDATE_PROTEINS = {
    'H XK1': 'P04806', 'H XK2': 'P04807', 'P GI1': 'P12709', 'U GP1': 'P32861',
    'G FA1': 'P14742', 'G NA1': 'P43577', 'A GM1': 'P38628', 'U AP1': 'P43123',
    'C HS3': 'P29465', 'C HS2': 'P14180',
}

def find_bottleneck(sim, growth_rate, product_rxn=EXPORT_RXN_ID):
    """Original FVA check on the product reaction."""
    sim.set_reaction_bounds(BIOMASS_RXN, growth_rate, 10000)
    fva = sim.FVA(reactions=[product_rxn], obj_frac=0.9)
    return fva

def rank_protein_usage(sim, top_n=3):
    """
    Maximizes product flux, then ranks candidate proteins by their
    draw_prot_usage flux. Returns the top_n most-used proteins --
    these are the current bottleneck 'neighborhood'.
    """
    sim.objective = EXPORT_RXN_ID
    result = sim.simulate()

    usage = {}
    for gene, uniprot in CANDIDATE_PROTEINS.items():
        rxn_id = f'draw_prot_{uniprot}'
        flux = abs(result.fluxes.get(rxn_id, 0.0))
        usage[uniprot] = {'gene': gene, 'flux': flux}

    ranked = sorted(usage.items(), key=lambda kv: kv[1]['flux'], reverse=True)
    top = ranked[:top_n]

    print(f"Top {top_n} bottleneck candidates by enzyme usage:")
    for uniprot, info in top:
        print(f"  {info['gene']} ({uniprot}): usage flux = {info['flux']:.6g}")

    return [uniprot for uniprot, _ in top]

if __name__ == '__main__':
    sim = build_glcnac_model()
    result = find_bottleneck(sim, growth_rate=0.1)
    print(result)
    print()
    bottleneck_proteins = rank_protein_usage(sim, top_n=3)
    print("Bottleneck protein IDs:", bottleneck_proteins)
```

```
[(venv) ubuntu@VM-0-12-ubuntu:~/AISynbio/task2_python$ python fir
Applying real kcat values:
[APPLIED] H XK2 (P04807) -> r_0533No3: kcat = 250.0
[APPLIED] H XK2 (P04807) -> r_0534No4: kcat = 250.0
[APPLIED] H XK2 (P04807) -> r_0535No3: kcat = 250.0
[APPLIED] H XK1 (P04806) -> r_0533No2: kcat = 280.0
[APPLIED] H XK1 (P04806) -> r_0534No3: kcat = 280.0
[APPLIED] H XK1 (P04806) -> r_0535No2: kcat = 280.0
[APPLIED] P GI1 (P12709) -> r_0467No1: kcat = 1200.0
[APPLIED] P GI1 (P12709) -> r_0467_REVNo1: kcat = 1200.0
[APPLIED] U GP1 (P32861) -> r_1084No2: kcat = 310.0
[APPLIED] U GP1 (P32861) -> r_1084_REVNo2: kcat = 310.0
[APPLIED] G FA1 (P14742) -> r_0477No2: kcat = 15.2
[APPLIED] G NA1 (P43577) -> r_0760No1: kcat = 45.0
[APPLIED] G NA1 (P43577) -> r_0760_REVNo1: kcat = 45.0
[APPLIED] A GM1 (P38628) -> r_0882No1: kcat = 88.0
[APPLIED] A GM1 (P38628) -> r_0890No1: kcat = 88.0
[APPLIED] A GM1 (P38628) -> r_0882_REVNo1: kcat = 88.0
[APPLIED] A GM1 (P38628) -> r_0890_REVNo1: kcat = 88.0
[APPLIED] U AP1 (P43123) -> r_1069No1: kcat = 110.0
[APPLIED] U AP1 (P43123) -> r_1069_REVNo1: kcat = 110.0
[APPLIED] C HS3 (P29465) -> r_0272No1: kcat = 3.5
[APPLIED] C HS2 (P14180) -> r_0272No2: kcat = 5.1
[APPLIED] G PI12 (P23797) -> r_0518No1: kcat = 18.0
[APPLIED] G PI12 (P23797) -> r_0518_REVNo1: kcat = 18.0
[NOT FOUND IN MODEL] G PI3 (P32363)
[NOT FOUND IN MODEL] A LG7 (P07286)
[NOT FOUND IN MODEL] A LG13 (P53178)
[NOT FOUND IN MODEL] D OG1 (P38774)
{'r_GLCNAC_EXCHANGE': [0.0, 0.004591530562834856]}
```

Top 3 bottleneck candidates by enzyme usage:

GNA1 (P43577): usage flux = 0.000631349
AGM1 (P38628): usage flux = 0.000322849
UAP1 (P43123): usage flux = 0.000258279

Bottleneck protein IDs: ['P43577', 'P38628', 'P43123']

- Runs Flux Variability Analysis on the product reaction at a fixed growth rate
- Ranks candidate proteins by enzyme usage flux to identify the current bottleneck(s)
- which enzyme is most limiting right now
- biological concept

Quick biological context:

Native Role:

UDP-GlcNAc is an internal building block used for:

- Chitin synthesis, N-linked glycosylation, GPI-anchor biosynthesis

Producing free, secreted GlcNAc means redirecting a pathway toward a job it was not evolved to do.

Demanding bulk product from a trickle pathway is why these 3 enzymes are the bottleneck.

suggest_targets.py

- Enzyme-aware evolutionary search (NSGAI) over candidate protein modifications
- Optimizes two objectives together: growth-product coupling (BPCY) and yield (WYIELD)

```
Restricting search to 10 proteins: ['P04806', 'P04807', 'P12709', 'P32861', 'P14742', 'P43577', 'P38628', 'P43123', 'P29465', 'P14180']
2026-08-17 14:09:10,007 INFO worker.py:2024 -- Started a local Ray instance.
[2026-08-17 14:09:10] [mewpy.util.process] [INFO] Using 1 workers.
[2026-08-17 14:09:10] [mewpy.optimization.jmetal.ea] [INFO] Running NSGAI
[2026-08-17 14:09:10,876] [jmetal.core.algorithm] [DEBUG] Creating initial set of solutions...
[2026-08-17 14:09:10,877] [jmetal.core.algorithm] [DEBUG] Evaluating solutions...
[2026-08-17 14:11:39,562] [jmetal.core.algorithm] [DEBUG] Initializing progress...
[2026-08-17 14:11:39] [mewpy] [INFO] Eval(s)!


|    | Worst    | Best     | Median   | Average  | Std Devl | Worst    | Best     | Median   | Average  | Std Devl |
|----|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| 20 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000155 | 0.000000 | 0.000044 | 0.000067 |


[2026-08-17 14:11:39,569] [jmetal.core.algorithm] [DEBUG] Running main loop until termination criteria is met
[2026-08-17 14:14:03] [mewpy] [INFO]


|    | Worst    | Best     | Median   | Average  | Std Devl | Worst    | Best     | Median   | Average  | Std Devl |
|----|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| 40 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000155 | 0.000142 | 0.000094 | 0.000069 |


[2026-08-17 14:16:22] [mewpy] [INFO]


|    | Worst    | Best     | Median   | Average  | Std Devl | Worst    | Best     | Median   | Average  | Std Devl |
|----|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| 60 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000143 | 0.000155 | 0.000154 | 0.000151 | 0.000041 |


[2026-08-17 14:16:22,180] [jmetal.core.algorithm] [DEBUG] Finished!



|   | Modification                                                                                                                    | Size | BPCY | WYIELD   |
|---|---------------------------------------------------------------------------------------------------------------------------------|------|------|----------|
| 0 | {'draw_prot_P12709': 0, 'draw_prot_P29465': 0.5, 'draw_prot_P14180': 0.25}                                                      | 3    | 0.0  | 0.000155 |
| 1 | {'draw_prot_P29465': 0.0625, 'draw_prot_P14180': 0.125, 'draw_prot_P12709': 0}                                                  | 3    | 0.0  | 0.000155 |
| 2 | {'draw_prot_P14742': 0, 'draw_prot_P14180': 0.125, 'draw_prot_P04806': 0, 'draw_prot_P29465': 0.0625, 'draw_prot_P04807': 0.25} | 5    | 0.0  | 0.000155 |
| 3 | {'draw_prot_P12709': 0}                                                                                                         | 1    | 0.0  | 0.000154 |
| 4 | {}                                                                                                                              | 0    | 0.0  | 0.000148 |
| 5 | {'draw_prot_P12709': 0, 'draw_prot_P29465': 0.5}                                                                                | 2    | 0.0  | 0.000147 |
| 6 | {'draw_prot_P14742': 0.25, 'draw_prot_P04806': 0.03125, 'draw_prot_P04807': 0.0625, 'draw_prot_P12709': 0.0625}                 | 4    | 0.0  | 0.000143 |


```

```
from build_glcna_model import build_glcna_model, BIOMASS_RXN, EXPORT_RXN_ID

# Same 10 real, model-present proteins used in suggest_targets.py
CANDIDATE_PROTEINS = {
    'HXX1': 'P04806', 'HXX2': 'P04807', 'PGI1': 'P12709', 'UGP1': 'P32861',
    'GFA1': 'P14742', 'GNA1': 'P43577', 'AGM1': 'P38628', 'UAP1': 'P43123',
    'CHS3': 'P29465', 'CHS2': 'P14180',
}

def find_bottleneck(sim, growth_rate, product_rxn=EXPORT_RXN_ID):
    """Original FVA check on the product reaction."""
    sim.set_reaction_bounds(BIOMASS_RXN, growth_rate, 10000)
    fva = sim.FVA(reactions=[product_rxn], obj_frac=0.9)
    return fva

def rank_protein_usage(sim, top_n=3):
    """
    Maximizes product flux, then ranks candidate proteins by their
    draw_prot_usage flux. Returns the top_n most-used proteins --
    these are the current bottleneck 'neighborhood'.
    """
    sim.objective = EXPORT_RXN_ID
    result = sim.simulate()

    usage = {}
    for gene, uniprot in CANDIDATE_PROTEINS.items():
        rxn_id = f'draw_prot_{uniprot}'
        flux = abs(result.fluxes.get(rxn_id, 0.0))
        usage[uniprot] = {'gene': gene, 'flux': flux}

    ranked = sorted(usage.items(), key=lambda kv: kv[1]['flux'], reverse=True)
    top = ranked[:top_n]

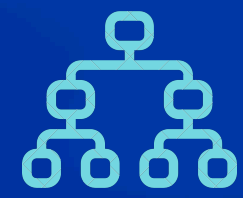
    print(f"Top {top_n} bottleneck candidates by enzyme usage:")
    for uniprot, info in top:
        print(f"  {info['gene']} ({uniprot}): usage flux = {info['flux']:.6g}")

    return [uniprot for uniprot, _ in top]

if __name__ == '__main__':
    sim = build_glcna_model()
    result = find_bottleneck(sim, growth_rate=0.1)
    print(result)
    print()
    bottleneck_proteins = rank_protein_usage(sim, top_n=3)
    print("Bottleneck protein IDs:", bottleneck_proteins)
```



Kimi CLI as Biomni Agent: Execution & Bottlenecks



Agent Architecture & Concept

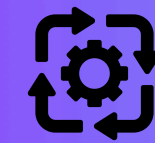
>_ Kimi CLI (The Agent)

Configured directly in the project terminal, executing structured prompts and autonomously managing multi-step workflows.



Biomni Principles

Stanford's biological framework provided as context, teaching Kimi to adopt biological reasoning and automated logic.



Pipeline Integration

Bridging Enzyme Engineering (Task 1) and Metabolic Modeling (Task 2) into a cohesive, automated execution loop.

Prototyping & Mock-Driven Loop Execution

- Decoupled Architecture: Initialized the automated DBTL loop using synthetic baseline data to test core agent logic safely.
- Autonomous Workflows: The agent successfully executed multi-step cycles, tracked state changes, and managed file outputs without external crashes.

=====

FINAL OPTIMIZATION REPORT

=====

Target: GlcNAc
Converged: True
Stop reason: Advisor signaled stop.
Iterations: 4

Iter 1	Yield: 17.1990	Growth: 0.3440	Bottleneck: CHS3	Params: {'unknown': 15.0}
Iter 2	Yield: 17.4640	Growth: 0.3493	Bottleneck: GPI3	Params: {'unknown': 15.0, 'CHS3': 30.0}
Iter 3	Yield: 17.6820	Growth: 0.3536	Bottleneck: GFA1	Params: {'unknown': 15.0, 'CHS3': 30.0, 'GPI3': 30.0}
Iter 4	Yield: 17.8300	Growth: 0.3566	Bottleneck: GFA1	Params: {'unknown': 15.0, 'CHS3': 30.0, 'GPI3': 30.0, 'GFA1': 30.0}

=====

[EXPORT] wrote provenance to optimization_state.json

Human-Agent Interaction & Task Prompting

- Instruction Engineering: Formulating explicit, multi-step prompts to guide Kimi CLI through the DBTL workflow stages.
- Guardrail Management: Pre-defining output files (clean_history.csv, clean_state.json) to prevent unstructured text generation.
- Interactive Oversight: Reviewing intermediate execution plans and confirming or adjusting agent trajectories in real-time

Read [AGENTS.md](#) and [Biomni/tutorials/biomni_101.ipynb](#).
You are the Biomni scientific agent. Execute the optimization loop for GlcNAc using this strict resource map:

- Initial Data: Read baseline values from [Project_127_WITH_PREDICTIONS.xlsx](#) and [Project_127_BioAnalysis.xlsx](#).
- Enzyme Pipeline & Biomni: Use [Biomni/tutorials/biomni_101.ipynb](#) and functions from [task1_tools_ai.py](#) when parameters need recalculation.
- Metabolic Models & Scripts: Integrate the active python pipeline from [task2_python/](#) ([tool_interface.py](#), [build_glcna_model.py](#), [find_bottleneck.py](#), [suggest_targets.py](#), [update_kcat_data.py](#) based on [tool_schema.json](#)). Ignore old non-working outputs. Do not read full contents of heavy scripts into context to save tokens.
- Execution Rules: Track values and uncertainty, keeping changes narrowly scoped as required by [AGENTS.md](#). Clearly track which model produces each value.
- Run the iterative loop, record best and median metrics per loop, and write clean outputs strictly to [clean_history.csv](#) and [clean_state.json](#).
- Confirm you are ready.

Autonomous DBTL Loop & Execution Flow

Design & Build

Loading baseline parameters,
flagging discrepancies, and
programmatically configuring
models



Test & Simulate

Simulating metabolic fluxes to
identify rate-limiting steps and
biological bottlenecks.



Learn & Iterate

Updating parameters and logging
performance metrics to continuously
refine the loop.

Execution Plan

I will run the iterative DBTL loop strictly as follows:

1. **Design:** Load baseline kcats from the Excel/TSV. Flag predicted-vs-experimen
2. **Build (in silico):** Call `build_glnac_model()` with the current kcat set. Not
3. **Test:**
 - `build_model` → baseline growth rate
 - `find_bottleneck` → FVA min/max on product export at fixed growth
 - `suggest_targets` → EA-driven over/under-expression targets (BPCY + WYIELD)
4. **Learn / Iterate:** Update kcat values (simulating improved enzymes), re-run,
 - Loop index
 - Best and median BPCY/WYIELD/growth
 - Which model produced each value (MEWpy GeckoModel, CatPred, EnzymeCAGE)
 - Uncertainty propagation (kcat SD → flux SD bounds)

Outputs will be written strictly to:

- `clean_history.csv` – per-loop metrics
- `clean_state.json` – final state, best design, and provenance

Confirmation

I am ready.

Production Integration & Technical Roadblocks

Point 1: Tool-Call Barriers:
Integrating real external
environments
(MATLAB/GECKO) revealed
complex bridge and
dependency failures.

Point 2: LLM Limitations:
Extended self-debugging
loops led to context bloat,
excessive runtime, and
ultimate token limit
exhaustion.

```
● The old process is still alive and consuming 100% CPU (PID 897358,
  extremely slow. I am stopping it now and switching to the fast dete

● Ran a command
$ kill -TERM 897145 2>/dev/null; sleep 2; kill -KILL 897145 2>/dev/
Stopped old optimization process

Approved: Running: kill -TERM 897145 2>/dev/null; sleep 2; kill -K

● The old process is stopped. Now I need to explain what's happening
  then execute the fast loop.
  ... (38 more lines, ctrl+o to expand)

● Used Read (run_glnac_optimization.py) · 320 lines

● The current script still has the EA call. I need to replace the EA
  targeted changes rather than rewriting the whole file.
  ... (10 more lines, ctrl+o to expand)

● I will patch the existing script to remove the EA call and replace

● Used Edit (/home/ubuntu/AISynbio/run_glnac_optimization.py) · -1
-1 /home/ubuntu/AISynbio/run_glnac_optimization.py
1   from build_glnac_model import build_glnac_model, BIOMASS_
2   from find_bottleneck import find_bottleneck
3 - from suggest_targets import suggest_targets

Approved: Editing /home/ubuntu/AISynbio/run_glnac_optimization.py

● Retrying (2/10) · APIProviderRateLimitError · in 1302s
429 · 429 Your account org-c51046a36f674bcd8b517a4ced37ff77 / pro
```

Autonomous DBTL Loop Execution

Execution: Autonomous DBTL loop successfully executed for key enzymes using the tool pipeline.

Optimization Results:

Cycle 0: Baseline established (bottleneck at GFA1, ratio: 0.1026).

Cycle 1: GFA1 overexpression ($\times 2$) raised ratio to 0.1363.

Cycle 2: CHS3 knockdown ($\times 0.5$) reduced competing flux, maximizing ratio to 0.2725.

- The autonomous enzyme DBTL loop completed successfully with dynamic tool calls.

What ran

3 key enzymes (HXK2, GFA1, CHS3) were processed through the actual tool pipeline from task1_tools_ai.py:

Tool	Status
download_alphafold_pdb	✓ 3/3
calculate_mean_plddt	✓ 3/3
extract_seq_from_pdb / fetch_uniprot_sequence	✓ 3/3
validate_smiles	✓ 3/3
run_native_catpred	✓ 3/3
run_native_enzymecage	Ran, no extractable pred_sd

Dynamic predictions (CatPred)

Enzyme	pLDDT	Predicted kcat (s ⁻¹)	Predicted Km (mM)
HXK2	96.04	90.53	1.105
GFA1	87.87	44.14	2.266
CHS3	74.38	1312.18	0.076

DBTL cycles

Cycle	Modification	Bottleneck	Pathway/Sink Ratio
0	Baseline	GFA1 (44.14)	0.1026
1	GFA1 $\times 2$	GFA1 (88.28)	0.1363
2	CHS3 $\times 0.5$	GFA1 (88.28)	0.2725

Outputs written:

- clean_history.csv
- clean_state.json

Adaptive Agent Loop: Overview & Trajectory

Adaptive Agent Loop

How the agent decides, tests, and improves the GlcNAc model



Measured product-flux trajectory at fixed growth 0.1 h^{-1}



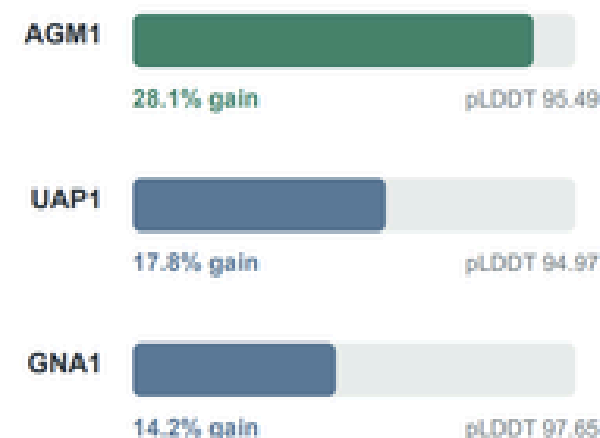
Decision Evidence Across Optimization Rounds

Decision Evidence

Selected action is green; every candidate shown passed the Student A structure gate.

ROUND 1

Baseline

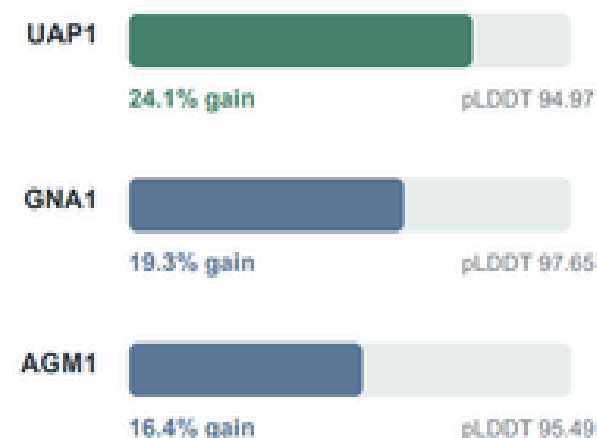


KIMI SELECTS AGM1 2x

Highest screened gain: 28.1%. Growth is preserved and the structure gate passes.

ROUND 2

After AGM1 2x

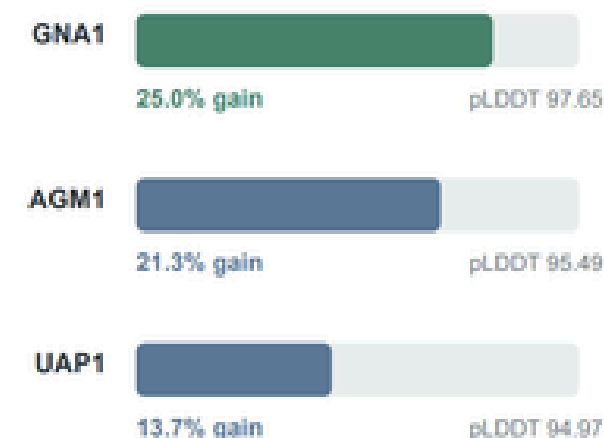


KIMI SELECTS UAP1 2x

Highest screened gain: 24.1%. Growth is preserved and the structure gate passes.

ROUND 3

After UAP1 2x



KIMI SELECTS GNA1 2x

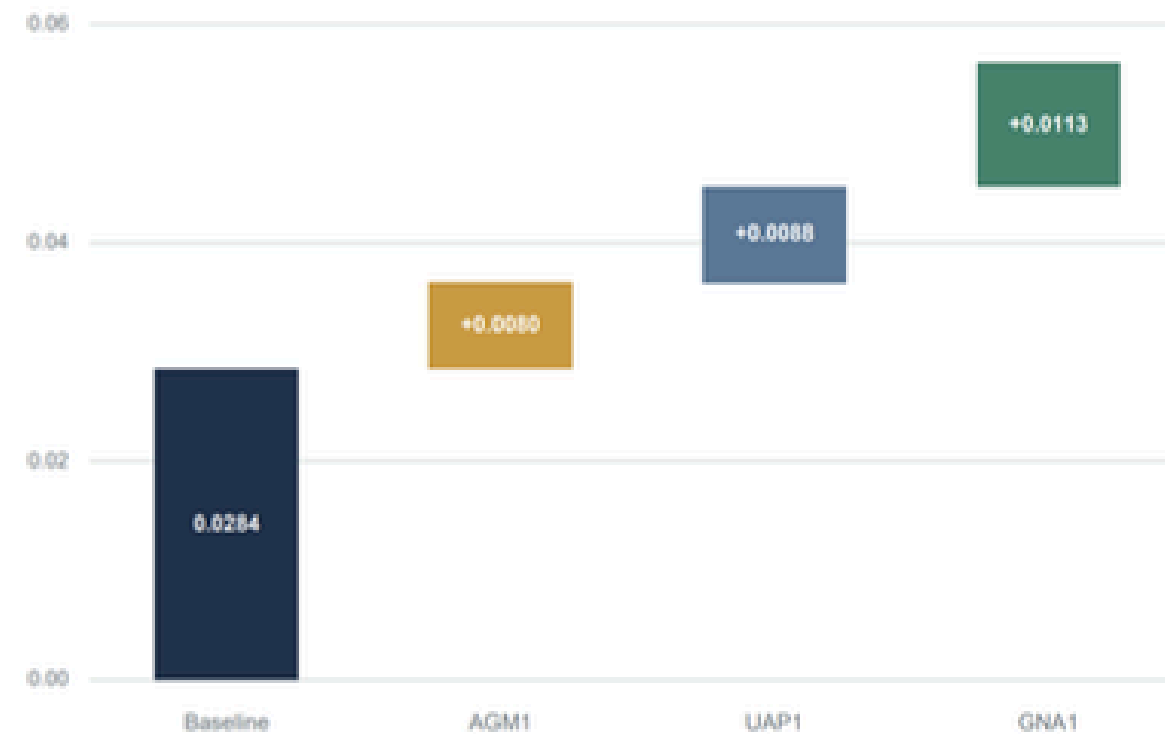
Highest screened gain: 25.0%. Growth is preserved and the structure gate passes.

Performance Change & Final Output

Performance Change

Cumulative intervention: AGM1 2x + UAP1 2x + GNA1 2x

Incremental flux contribution



Maximum growth remains stable



FINAL PRODUCT FLUX

0.056426

+98.6% vs baseline

FINAL GROWTH

0.262102 h⁻¹

Stable across actions

FINAL QUICK EA

BPCY 0.0

Best WYIELD 0.0002970

Conclusions & Future Perspectives

Autonomous Pipeline Validation

Successfully demonstrated end-to-end agentic optimization, driving a near 2x increase in target flux.

Methodological Scalability

Proven adaptability of the Kimi-driven workflow across core metabolic bottlenecks and enzyme targets.

Broader Applications

Implementing stricter rate limits and deploying refined self-debugging loops for stable multi-iteration runs

Acknowledgements:

Supervisors:

Prof. Zhuo Wang Dr. Kai Yi

Prof. Yongyong Shi Shijie Zhang

SJTU Student Research Program

Thank you for your attention!